

Module 9: AI Agent Observability and AgentOPs

Demo 2

Demo: AgentOps Practical Implementation

NOTE: This Demonstration has been implemented on [Google Colab](#).

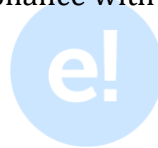
Overview

This documentation demonstrates the practical implementation of AgentOps observability platform integrated with a CrewAI-based stock analysis workflow. While the core stock analysis demonstration was covered in Module 7, this implementation focuses specifically on adding comprehensive monitoring, logging and observability to agent systems using AgentOps.

Scenario

A financial technology company needs to deploy an automated stock analysis system that can provide real-time insights to investment advisors. However, they require full visibility into their AI agents' performance, decision-making processes and operational metrics to ensure reliability and compliance with financial regulations.

Problem Statement



When deploying multi-agent systems in production environments, organizations face several critical challenges:

- **Lack of Visibility:** Traditional AI workflows operate as "black boxes" with no insight into agent interactions, decision paths, or performance metrics
- **Debugging Complexity:** When multi-agent systems fail or produce unexpected results, it's difficult to identify which agent, task, or interaction caused the issue
- **Performance Monitoring:** Without proper observability, teams cannot track response times, success rates, or resource utilization across different agents
- **Compliance Requirements:** Financial applications require detailed audit trails and logging for regulatory compliance

Solution: AgentOps Integration

AgentOps provides comprehensive observability for AI agent workflows by offering:

- **Real-time Monitoring:** Track agent execution, task completion and system performance in real-time
- **Detailed Logging:** Capture every agent interaction, tool usage and decision point
- **Performance Analytics:** Monitor response times, success rates and resource utilization
- **Visual Workflow Tracking:** See how agents collaborate and tasks flow through the system
- **Error Tracking:** Identify and debug issues with detailed error logs and stack traces

Setup and Installation

Prerequisites

- Python 3.8 or higher
- Google Gemini API Key
- AgentOps API Key (sign up at agentops.ai)

Installation Steps

Open and run the Jupyter Notebook:

Open `AgentOps\_Practical\_Implementation.ipynb` in Google Colab.

Run all the cells in the notebook.

Install Dependencies The following command installs all required packages including AgentOps integration:

```
pip install crewai crewai-tools agentops google-generativeai duckduckgo_search yfinance pandas beautifulsoup4 python-dotenv
```

Key Dependencies Explained:

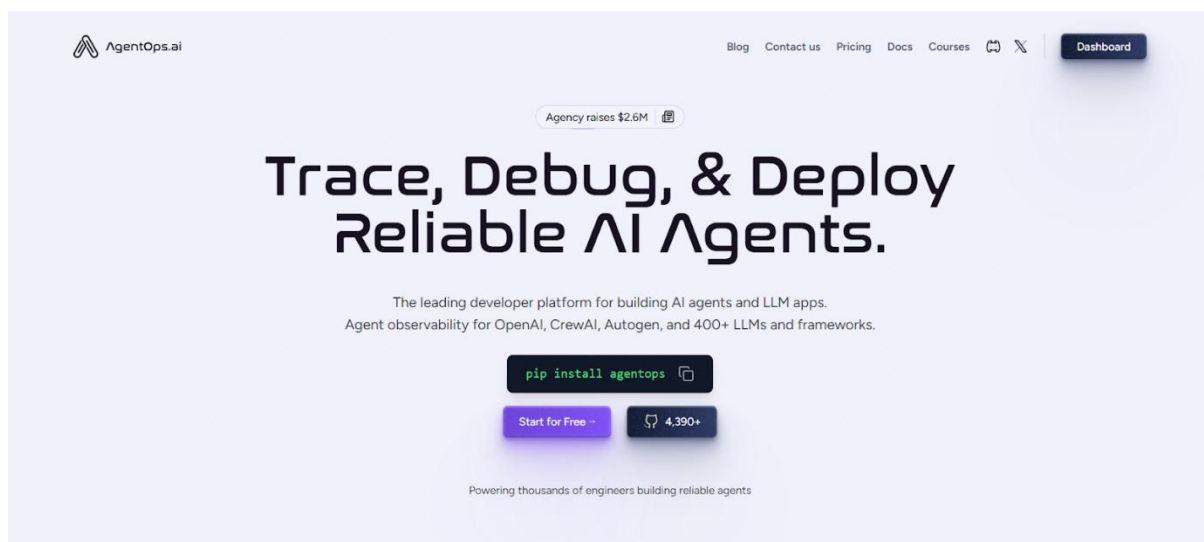
- **agentops:** Core observability platform for AI agents

- **crewai**: Multi-agent orchestration framework
- **google-generativeai**: Gemini LLM integration
- **yfinance**: Financial data retrieval
- **duckduckgo_search**: Web search capabilities

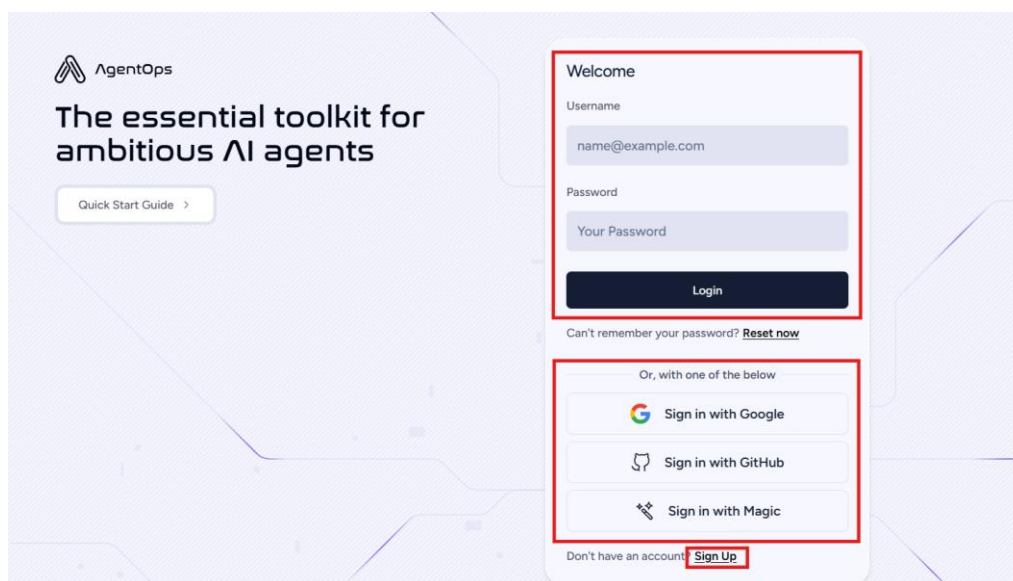
Steps to Generate AgentOps API Key

1. Visit the AgentOps Website

Go to <https://www.agentops.ai>



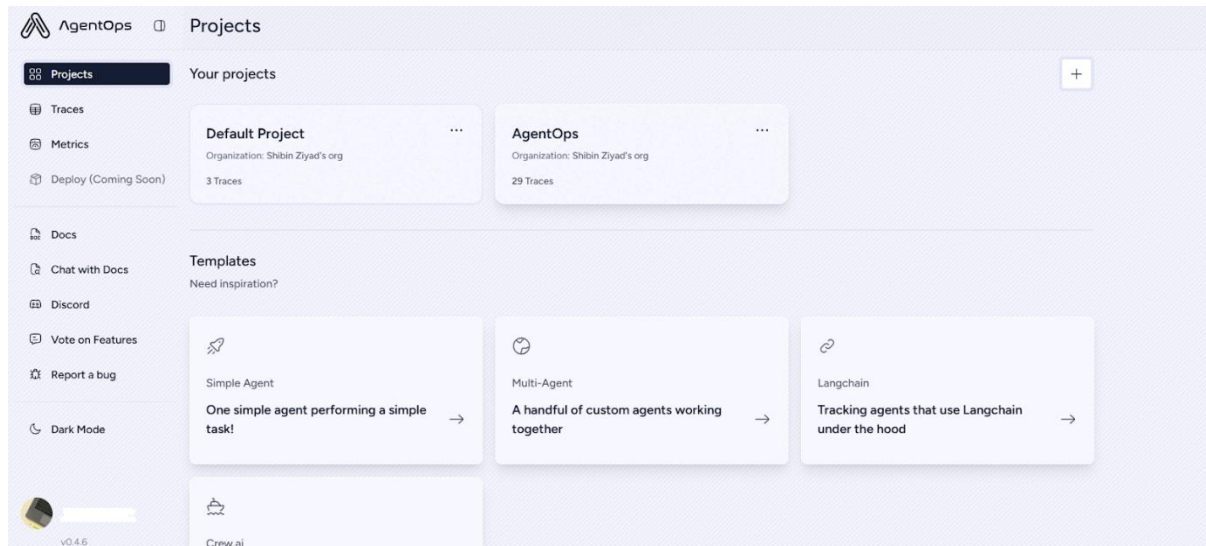
2. Sign Up or Log In with you mail.



Use your Google account or another supported login method to access your dashboard.

3. Access the AgentOps Dashboard

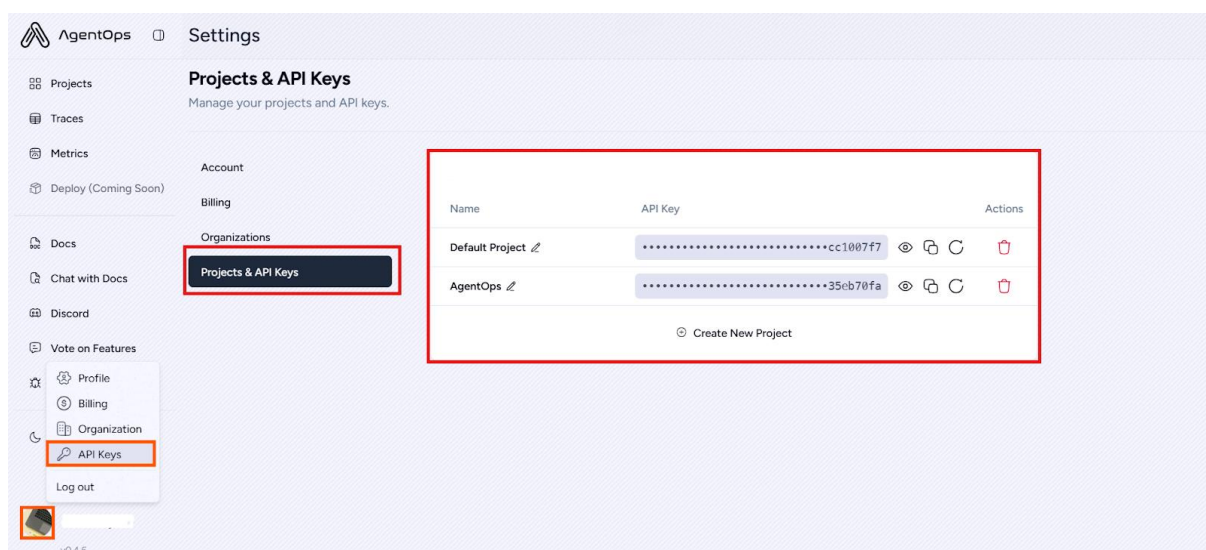
Once logged in, you'll be redirected to the AgentOps dashboard.



This is where you can monitor your agents and manage your settings.

4. Navigate to API Keys

Navigate to API Keys from the profile settings, find and click on “API Keys” and then click on “Projects & API Keys”



Environment Setup Create a .env file for secure API key storage:

1. GEMINI_API_KEY=your_gemini_api_key_here
2. AGENTOPS_API_KEY=your_agentops_api_key_here

Implementation Steps

STEP 1: Install Dependencies

```
pip install crewai crewai-tools agentops google-  
generativeai duckduckgo_search yfinance pandas  
beautifulsoup4 python-dotenv
```

Step 2: Import Dependencies with AgentOps



```
1 import os  
2 from dotenv import load_dotenv  
3 from getpass import getpass  
4 import yfinance as yf  
5 from duckduckgo_search import DDGS  
6 from crewai import Agent, Task, Crew, LLM  
7 from crewai.tools import BaseTool  
8 import agentops  
9  
10 # Load environment variables  
11 load_dotenv()
```

AgentOps Integration Point: The import agentops statement brings in the observability framework that will monitor all agent activities. This import is crucial as it enables the tracking capabilities throughout the entire workflow.

Step 3: API Key Configuration and AgentOps Initialization

```
1 # Load Gemini API Key
2 os.environ["GEMINI_API_KEY"] = getpass("Enter your Gemini API Key: ")
3
4 # Load AgentOps API Key
5 os.environ["AGENTOPS_API_KEY"] = getpass("Enter your AgentOps API Key: ")
6
7 # Initialize AgentOps
8 agentops.init(api_key=os.environ["AGENTOPS_API_KEY"])
9
```

AgentOps Functionality:

- `agentops.init()` establishes connection to the AgentOps platform
- Creates a new monitoring session that will track all subsequent agent activities
- Returns a session object that manages the connection and data flow
- All agent interactions, tool usage and performance metrics will be automatically captured from this point forward

Step 4: LLM Configuration

```
1 llm = LLM(
2     model="gemini/gemini-2.0-flash",
3     api_key=os.environ["GEMINI_API_KEY"],
4     temperature=0.7
5 )
6
```

AgentOps Monitoring: Once AgentOps is initialized, it automatically tracks LLM usage including token consumption, response times and model performance metrics.

Step 5: Custom Tool Definition

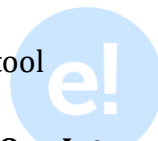
```

1 class StockSearchTool(BaseTool):
2     name: str = "StockNewsSearcher"
3     description: str = "Search for the latest news and updates about a stock using DuckDuckGo"
4
5     def _run(self, query: str) -> str:
6         with DDGS() as ddgs:
7             results = ddgs.text(query, max_results=2)
8             return "\n".join([r['body'] for r in results])
9
10 class YahooFinanceTool(BaseTool):
11     name: str = "YahooFinanceFetcher"
12     description: str = "Get the latest 1-month stock price history for a given ticker using yFinance"
13
14     def _run(self, ticker: str) -> str:
15         stock = yf.Ticker(ticker)
16         hist = stock.history(period="1mo")
17         return hist.tail(3).to_string()
18

```

AgentOps Tool Tracking: AgentOps automatically monitors:

- Tool execution frequency and duration
- Input parameters and output results
- Success/failure rates for each tool



Step 6: Agent Definition with AgentOps Integration

```

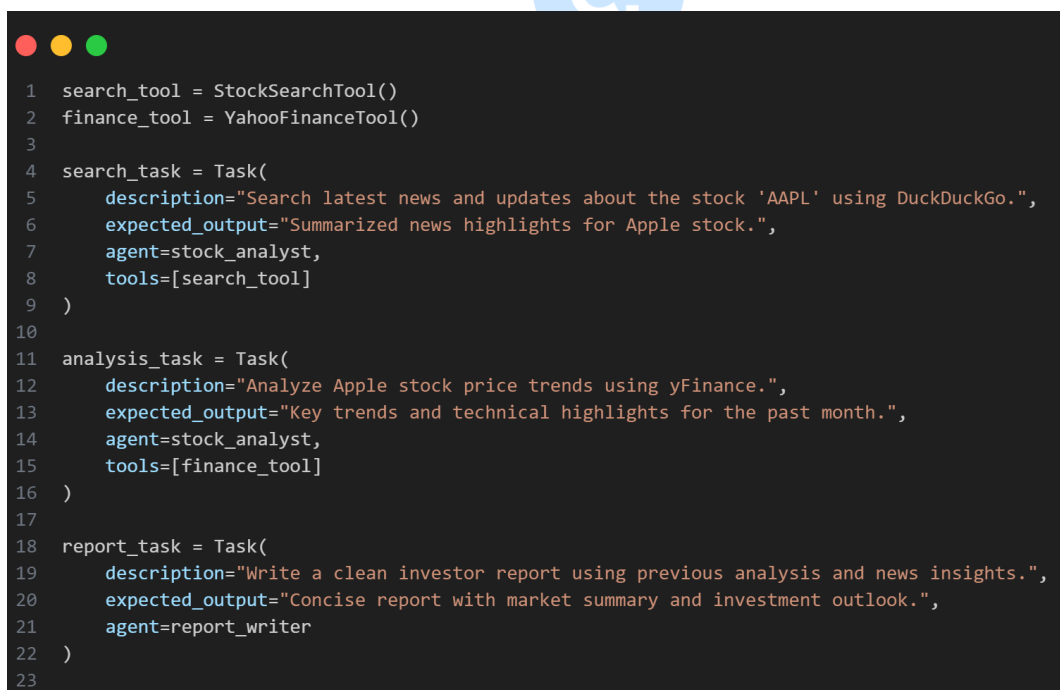
1 stock_analyst = Agent(
2     role='Stock Analyst',
3     goal='Analyze recent stock data and news',
4     backstory='Expert in financial trends, macro indicators, and company performance',
5     verbose=True,
6     allow_delegation=False,
7     llm=llm,
8     agentops_enabled=True # ✅ Enables AgentOps logging
9 )
10
11 report_writer = Agent(
12     role='Report Generator',
13     goal='Write investor-friendly summaries of stock analysis',
14     backstory='Professional writer with expertise in finance reporting',
15     verbose=True,
16     allow_delegation=False,
17     llm=llm,
18     agentops_enabled=True # ✅ Enables AgentOps logging
19 )
20

```

Critical AgentOps Feature: The `agentops_enabled=True` parameter is the key integration point that enables comprehensive monitoring for each agent. This setting ensures:

- **Agent Lifecycle Tracking:** Monitor when agents start, complete tasks, or encounter errors
- **Decision Path Logging:** Track the reasoning process and decision points of each agent
- **Interaction Mapping:** Record how agents communicate and collaborate
- **Performance Metrics:** Measure response times, task completion rates and resource usage per agent
- **Conversation History:** Maintain detailed logs of all agent interactions and outputs

Step 7: Task Configuration

A code editor window with a dark background and light-colored text. It contains Python code for configuring tasks. The code defines three tools: StockSearchTool, YahooFinanceTool, and a Task object for each. The tasks are: search_task (searching for AAPL stock news), analysis_task (analyzing Apple stock price trends), and report_task (writing an investor report). The code is numbered 1 through 23.

```
1 search_tool = StockSearchTool()
2 finance_tool = YahooFinanceTool()
3
4 search_task = Task(
5     description="Search latest news and updates about the stock 'AAPL' using DuckDuckGo.",
6     expected_output="Summarized news highlights for Apple stock.",
7     agent=stock_analyst,
8     tools=[search_tool]
9 )
10
11 analysis_task = Task(
12     description="Analyze Apple stock price trends using yFinance.",
13     expected_output="Key trends and technical highlights for the past month.",
14     agent=stock_analyst,
15     tools=[finance_tool]
16 )
17
18 report_task = Task(
19     description="Write a clean investor report using previous analysis and news insights.",
20     expected_output="Concise report with market summary and investment outlook.",
21     agent=report_writer
22 )
23
```

AgentOps

Task Monitoring: Each task execution is tracked with:

- Task start and completion timestamps

- Agent assignment and tool usage
- Success/failure status and error details
- Task dependencies and execution order
- Output quality and expected vs actual results

Step 8: Crew Assembly

```
1 crew = Crew(  
2     agents=[stock_analyst, report_writer],  
3     tasks=[search_task, analysis_task, report_task],  
4     verbose=False # Keep output clean  
5 )  
6
```

AgentOps

Crew Tracking: The crew orchestration is monitored to provide:

- Workflow visualization showing task dependencies
- Agent collaboration patterns
- Overall crew performance metrics
- Bottleneck identification and optimization suggestions

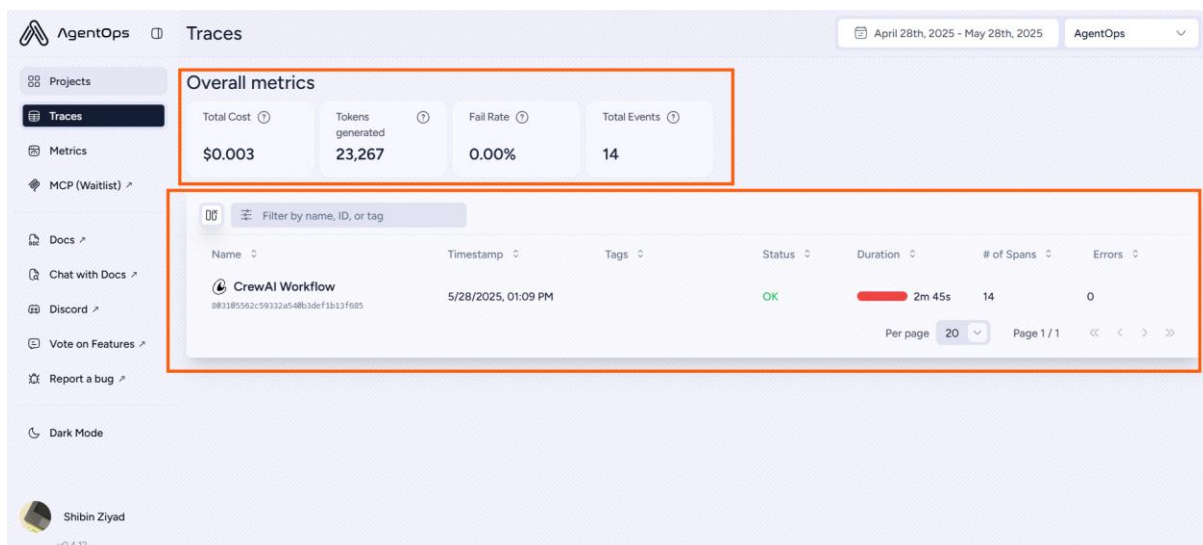
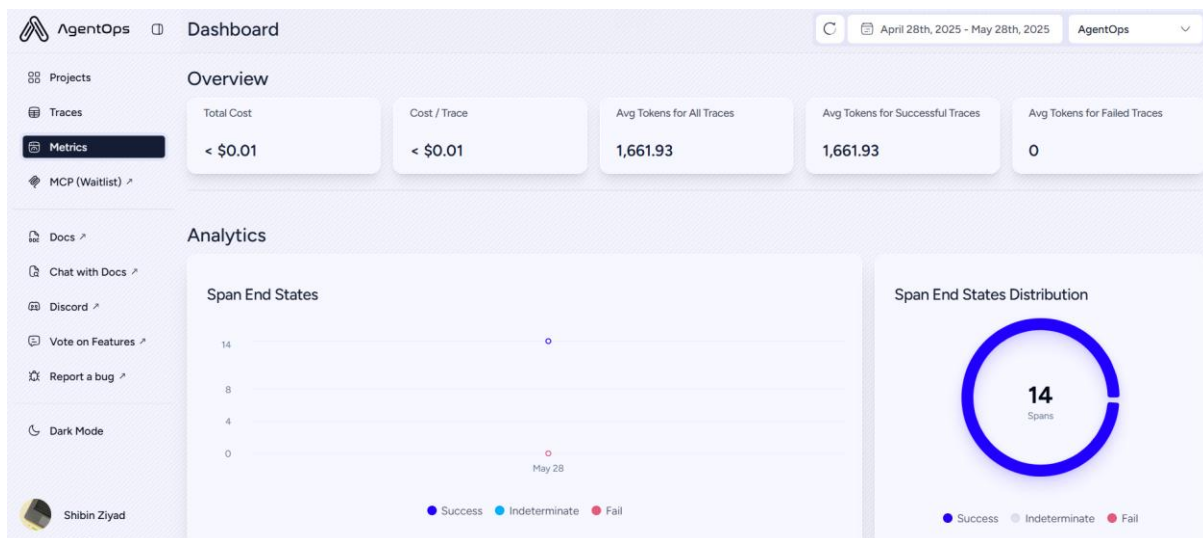
Step 9: Execution with Full Observability

```
1 result = crew.kickoff()  
2  
3 print("\n📊 Final Stock Analysis Report:\n")  
4 print(result)  
5
```

AgentOps Execution Monitoring: During the `crew.kickoff()` execution, AgentOps captures:

- **Real-time Progress:** Live updates on task execution status
- **Performance Metrics:** Response times, token usage and resource consumption
- **Error Handling:** Detailed error logs with stack traces and context
- **Success Tracking:** Completion rates and output quality assessments
- **Session Management:** Automatic session closure and data persistence

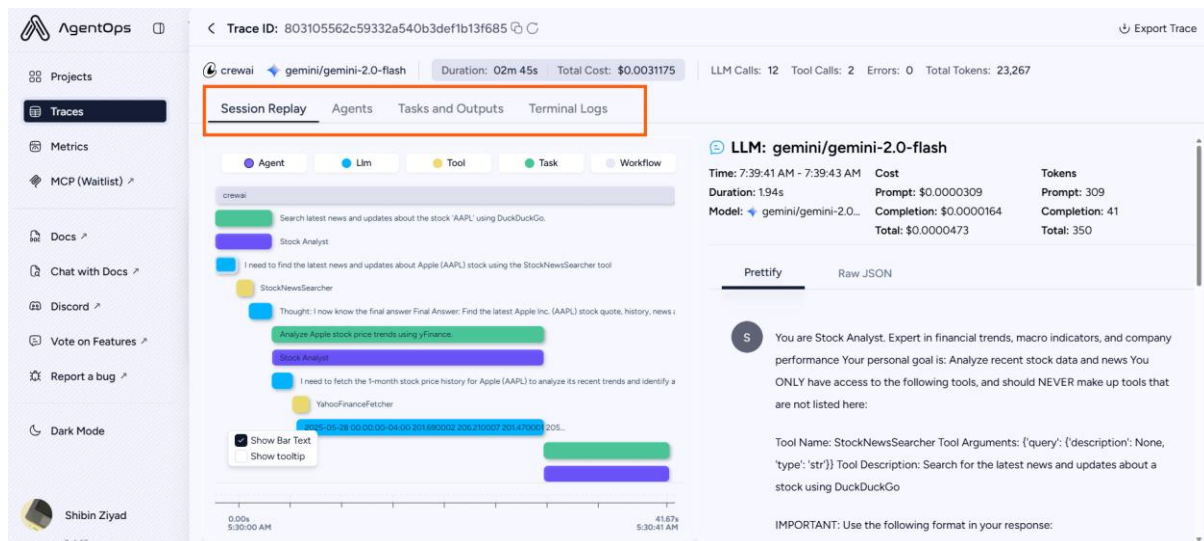
AgentOps Dashboard



Overall Metrics: Displays key performance indicators such as total cost, token usage, failure rate and number of trace events over the selected date range.

Trace List Entry: Shows detailed info for each workflow trace including name, timestamp, status, execution duration, number of spans and error count.

Traces in AgentOps



1. **Session Replay:** Visualize the step-by-step execution flow of an agent's actions, tools and decisions.
2. **Agents:** View and manage the autonomous agents involved in processing tasks and making decisions.
3. **Tasks and Outputs:** Track assigned tasks and inspect the corresponding generated outputs by the agents.
4. **Terminal Logs:** Access detailed backend logs for debugging and monitoring system-level operations during task execution.

This implementation showcases how AgentOps converts unclear AI agent processes into completely visible, tracked and enhanced systems. With the addition of a few lines of code (`import agentops`, `agentops.init()` and `agentops_enabled=True`), teams obtain in-depth insights into their multi-agent systems, facilitating dependable production deployments and ongoing enhancements